

6. Security Hotspots Analysis

Objective: Address key OWASP-class security vulnerabilities and dependency risk.

Date: July 15, 2026 | **Scope:** shende-shweta/FSDKC@main — Laravel 12 (PHP 8.3) API, React 19 / Vite 6 / TypeScript SPA, Node.js Express dev-api , Docker Compose (nginx, MariaDB 11, MongoDB 7)

Executive Summary

EXECUTIVE SUMMARY

Security review covered **backend** (18 PHP application files, 6 API controllers, 19 public Laravel routes), **frontend** (15 TS/TSX/JSX source files), and **dev-api** (6 JS files, 18 Express routes) fetched from shende-shweta/FSDKC@main via GitHub REST API and raw content scan. The stack uses Eloquent ORM and the MongoDB PHP driver with parameterized queries — no SQL, NoSQL, or shell injection hotspots were observed. The dominant risk is **complete absence of authentication and authorization** on every API route (Laravel and dev-api), compounded by **wildcard CORS** (allowed_origins: ['*'] / cors() with no origin filter), enabling any website or anonymous client to create jobs, start tests, read transcripts, and import monitors. A **session stream IDOR** lets any caller replay SSE events with only a session_id (the route {id} is ignored). PHP extract(\$request->all()) in LegacyReportController creates variable-scope injection from query parameters. Frontend XSS sinks, client-side secrets, and browser token storage were not observed; npm audit reported zero CVEs across frontend and dev-api lockfiles. CI runs PHPUnit and frontend build only — no npm audit, composer audit, or SAST gate. Overall verdict: **High Risk**, driven by one Critical finding (missing API authentication), three High findings (permissive CORS, session IDOR, unauthenticated bulk-import), and 9/12 OWASP categories with concrete findings.

39

FILES SCANNED FOR INPUT HANDLING

4

CONCRETE INJECTION/XSS/CSRF/CORS/AUTH FINDINGS

0

DEPENDENCIES FLAGGED OUTDATED/VULNERABLE

9/12

OWASP CATEGORIES WITH FINDINGS

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by Critical missing API authentication (H1), High permissive CORS and session stream IDOR (H2), and 9/12 OWASP categories with concrete findings (H5).

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	--------------	--------	------	----------	-----------	----------	--------

H1	Critical Vulnerabilities	0	0	1	>1	1 (unauthenticated API surface)	HIGH RISK
H2	High Vulnerabilities	0	<5	5–10	>10	3 (CORS wildcard, session IDOR, bulk-import)	GOOD
H3	Medium Vulnerabilities	low	<20	20–50	>50	7 (default creds, debug mode, missing headers, no rate limit, exposed DB ports, extract(), FS5 HTTP/CSP)	MODERATE
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	3.9/KLOC (11 findings / 2.8 KLOC)	HIGH RISK
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	25% categories clean (3/12)	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	0 (<code>npm audit</code> critical+high on frontend and dev-api)	GOOD
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	0% flagged (Laravel 12, React 19, Vite 6 — current majors)	GOOD
H8	End-of-Life Dependencies	0	0	1–5	>5	0 EOL majors detected	GOOD

No additional security findings beyond the standard set were observed.

6.2 Hotspot-by-Hotspot Evidence

Missing API Authentication CRITICAL

All 19 Laravel API routes in `backend/routes/api.php` and 18 Express routes in `dev-api/src/server.js` are publicly accessible with no `auth` middleware, Sanctum guard, JWT validation, or API-key check. Any anonymous HTTP client can list jobs, create monitors, start realtime tests, and read MongoDB transcripts.

Example 1 — `backend/routes/api.php:12-48`

```
Route::get('/health', function (MongoService $mongo) { ... });
Route::get('/mongodb/transcripts', [MongoController::class, 'transcripts']);
Route::post('/discovery/jobs', [DiscoveryController::class, 'store']);
Route::post('/discovery/jobs/{id}/start', [DiscoveryController::class, 'start']);
Route::post('/connect/monitors', [ConnectController::class, 'store']);
Route::post('/connect/monitors/{id}/run-check', [ConnectController::class, 'runCheck']);
```

Example 2 — backend/bootstrap/app.php:14-18

```
->withMiddleware(function (Middleware $middleware): void {
    $middleware->api(prepend: [
        \Illuminate\Http\Middleware\HandleCors::class,
    ]);
})
```

Only CORS middleware is prepended — no `auth:sanctum`, `Authenticate`, or custom API-key middleware.

Example 3 — dev-api/src/server.js:88-103

```
app.post('/api/discovery/jobs', (req, res) => {
    const job = { id: store.nextJobId++, name: req.body.name, ... };
    store.discoveryJobs.unshift(job);
    res.status(201).json({ data: job });
});
```

Exploit scenario: An attacker sends `POST /api/discovery/jobs` with arbitrary phone numbers and `POST /api/discovery/jobs/1/start` without credentials. The server creates and runs discovery jobs, writing test events and transcripts to MongoDB. Because CORS is wildcard (see below), a malicious website can trigger these calls from a victim's browser.

Recommended fix:

1. Add Laravel Sanctum or JWT middleware to `backend/bootstrap/app.php` and wrap all routes in `Route::middleware('auth:sanctum')`.
2. Add authorization policies on `DiscoveryController`, `ConnectController`, `MongoController`, and `StreamController` to enforce resource ownership.
3. Mirror auth on `dev-api` or remove the parallel unauthenticated runtime from production deployments.

No files matched `backend/routes/**/*.php` containing `Route::`.

Permissive CORS Configuration HIGH

Both API runtimes allow requests from any origin, enabling cross-site API abuse when combined with missing authentication.

Example 1 — backend/config/cors.php:3-11

```
return [
    'paths' => ['api/*', 'up'],
    'allowed_methods' => ['*'],
    'allowed_origins' => ['*'],
    'allowed_headers' => ['*'],
    'supports_credentials' => false,
];
```

Example 2 — dev-api/src/server.js:1,17

```
import cors from 'cors';
// ...
app.use(cors());
```

Default `cors()` accepts all origins with no environment-driven allow-list.

Exploit scenario: A malicious page at `https://evil.example` uses

`fetch('https://klearcom.example/api/discovery/jobs', { method: 'POST', body: ... })` from the victim's browser. Because `Access-Control-Allow-Origin: *` is returned and no auth is required, the request succeeds and creates resources on the platform.

Recommended fix:

1. Replace `allowed_origins: ['*']` in `backend/config/cors.php` with an env-driven allow-list (`CORS_ALLOWED_ORIGINS`).
2. Configure `cors({ origin: process.env.CORS_ALLOWED_ORIGINS?.split(',') })` in `dev-api/src/server.js`.
3. Set `supports_credentials: true` only when cookie-based auth is implemented and origins are explicit.

No files matched `**/*.php,js` containing `allowed_origins|app\.use\(|cors`.

Session Stream IDOR HIGH

SSE stream endpoints accept only `session_id` as a query parameter and ignore the route `{id}`. Any caller who learns or guesses a `session_id` can replay another user's realtime test events and transcripts.

Example 1 — `backend/app/Http/Controllers/Api/StreamController.php:16-29`

```
public function discoveryEvents(Request $request, int $id): StreamedResponse
{
    $sessionId = $request->string('session_id')->toString();
    abort_if($sessionId === '', 400, 'session_id required');
    return $this->streamSession($sessionId); // $id never validated
}
```

Example 2 — `backend/app/Services/MongoService.php:125-134`

```
public function getTestEvents(string $sessionId): array
{
    $cursor = $this->testEvents->find(
        ['session_id' => $sessionId],
        ['sort' => ['created_at' => 1]]
    );
}
```

Example 3 — `dev-api/src/server.js:135-138,242-248`

```
app.get('/api/discovery/jobs/:id/stream', (req, res) => {
  const sessionId = req.query.session_id;
  streamSession(res, req, sessionId); // req.params.id unused
});
```

Exploit scenario: After observing a `session_id` from a legitimate `POST /api/discovery/jobs/5/start` response (or brute-forcing UUIDs), an attacker calls `GET /api/discovery/jobs/999/stream?session_id=<stolen>` and receives the full SSE transcript stream regardless of job ownership.

Recommended fix:

1. Bind `session_id` to `resource_id` and authenticated user in `RealTimeTestService` at creation time.
2. Validate `$id` matches the session's bound resource in `StreamController::discoveryEvents` and `connectEvents`.
3. Apply the same binding check in `dev-api/src/server.js streamSession()`.

No files matched `backend/app/Http/Controllers/Api/StreamController.php` containing `streamSession|session_id`.

Unauthenticated Bulk Import HIGH

The dev-api exposes a bulk-import endpoint with no authentication, schema validation, size limits, or rate limiting.

Example 1 — `dev-api/src/server.js:143-161`

```
app.post('/api/connect/monitors/bulk-import', (req, res) => {
  // No validation, no rate limiting – accepts arbitrary body (security audit finding)
  const items = Array.isArray(req.body) ? req.body : req.body?.monitors ?? [];
  const created = items.map((item) => {
    const monitor = { id: store.nextMonitorId++, name: item.name ?? 'Imported',
    ... };
    store.connectMonitors.unshift(monitor);
    return monitor;
  });
  res.status(201).json({ data: created, imported: created.length });
});
```

Exploit scenario: An attacker sends `POST /api/connect/monitors/bulk-import` with a 10,000-element JSON array. The server accepts every item without auth, exhausting memory and polluting the monitor store — a DoS and data-integrity attack.

Recommended fix:

1. Remove `/api/connect/monitors/bulk-import` from production builds or gate it behind admin auth.
2. Add request body size limits, field schema validation (e.g., Zod/Joi), and `express-rate-limit`.
3. Log and alert on bulk-import usage.

No files matched `dev-api/**/*.js` containing `bulk-import`.

PHP extract() Variable Injection MEDIUM

Three `extract()` calls import user-controlled or untyped array keys into local variable scope, enabling variable-overwrite attacks and obscuring data flow.

Example 1 — `backend/app/Http/Controllers/Api/LegacyReportController.php:19-27`

```
$filters = $request->all();
extract($filters);
$monitors = ConnectMonitor::query()
    ->when(isset($country_code), fn ($q) => $q->where('country_code',
$country_code))
```

Example 2 — `backend/app/Legacy/LegacyDataMapper.php:10-18`

```
extract($row, EXTR_SKIP);
return ['label' => $name ?? 'Unknown', 'metric' => $reachability_pct ?? 0,
...];
```

Recommended fix:

1. Replace `extract($filters)` in `LegacyReportController` with explicit `$request->input('country_code')` access or a typed DTO.
2. Refactor `LegacyDataMapper` to use explicit array key access.

No files matched `backend/**/*.php` containing `extract\(\.`

Default Credentials and Debug Mode MEDIUM

Hardcoded database passwords and debug mode are committed in Docker Compose and environment templates.

Example 1 — `docker-compose.yml:19-27,36-40`

```
APP_DEBUG: "true"
DB_PASSWORD: secret
MYSQL_ROOT_PASSWORD: root
MYSQL_PASSWORD: secret
```

Example 2 — `backend/.env.example:4-12`

```
APP_DEBUG=true
DB_PASSWORD=secret
```

Recommended fix:

1. Remove hardcoded passwords from `docker-compose.yml`; use Docker secrets or `.env` files excluded from git.
2. Set `APP_DEBUG=false` in all non-local environments.

No files matched `**/*.{yml,example}` containing `DB_PASSWORD|APP_DEBUG|MYSQL_PASSWORD`.

Missing Security Headers MEDIUM

The nginx reverse proxy serves the Laravel API without CSP, HSTS, X-Frame-Options, or X-Content-Type-Options headers.

Example 1 — `docker/nginx/default.conf:1-17`

```
server {
    listen 80;
    # No add_header directives for security headers
```

Recommended fix:

1. Add `add_header X-Frame-Options "SAMEORIGIN" always;` and `add_header X-Content-Type-Options "nosniff" always;` to `docker/nginx/default.conf`.
2. Add HSTS when TLS termination is configured.

No files matched `docker/nginx/**/*.conf`.

No Rate Limiting MEDIUM

State-changing endpoints have no throttle middleware or express rate limiter.

Example 1 — `backend/routes/api.php:37,46`

```
Route::post('/jobs/{id}/start', [DiscoveryController::class, 'start']);
Route::post('/monitors/{id}/run-check', [ConnectController::class,
    'runCheck']);
```

Example 2 — `dev-api/src/server.js:144`

```
// No validation, no rate limiting – accepts arbitrary body (security audit finding)
```

Recommended fix:

1. Apply Laravel `throttle:60,1` middleware to POST routes in `backend/routes/api.php`.
2. Add `express-rate-limit` to `dev-api/src/server.js` on all POST endpoints.

No files matched `**/*.{php,js}` containing `Route::post|app\post\(|`.

Exposed Database Ports MEDIUM

MariaDB and MongoDB ports are published to the host in Docker Compose.

Example 1 — `docker-compose.yml:41-42,54-55`

```
ports:
  - "3306:3306"
  - "27017:27017"
```

Recommended fix:

1. Remove host port mappings from `docker-compose.yml`.
2. Keep database access on the internal Docker network only.

No files matched `docker-compose.yml` containing `"3306:3306" | "27017:27017"`.

FS1 — DOM/Stored/Reflected XSS Sinks CLEAN

Evidence: Not observed — scanned all 15 frontend source files for `dangerouslySetInnerHTML`, `innerHTML`, `outerHTML`, `document.write`, `eval()`, `new Function()`, and `v-html`. Zero matches. React 19 JSX auto-escapes interpolated values.

FS2 — Secrets / API Keys in Client Code CLEAN

Evidence: Not observed — scanned `frontend/src/**/*.{tsx,jsx,ts}` for hardcoded `apiKey`, `secret`, `Bearer`, `sk-`, `ghp_`, `AIza`. Only the public API base URL in `frontend/src/api/client.ts:1` (`VITE_API_URL` with localhost fallback) — no private keys.

FS3 — Auth Tokens in Browser Storage CLEAN

Evidence: Not observed — zero `localStorage` or `sessionStorage` usage in frontend source. API calls are unauthenticated `fetch` with no token persistence.

FS4 — Vulnerable / Outdated npm Dependencies CLEAN

`npm audit` on `frontend/package-lock.json` and `dev-api/package-lock.json` returned **zero** vulnerabilities. Major versions current: React 19.2.3, Vite 6.0.3, Laravel 12.

FS5 — Missing Frontend Security Controls MEDIUM

The SPA lacks Content-Security-Policy and defaults to HTTP for API calls.

Example 1 — `frontend/index.html:1-15`

```
<head>
  <meta charset="UTF-8" />
  <!-- No Content-Security-Policy meta tag -->
```

Example 2 — `frontend/src/api/client.ts:1`

```
const API_BASE = import.meta.env.VITE_API_URL ?? 'http://localhost:8080/api';
```


Recommended fix:

1. Add CSP via meta tag in `frontend/index.html` or nginx headers.
2. Enforce HTTPS `VITE_API_URL` in production Docker builds.

No files matched `frontend/**/*.{ts,tsx,html}` containing `VITE_API_URL|http://localhost`.

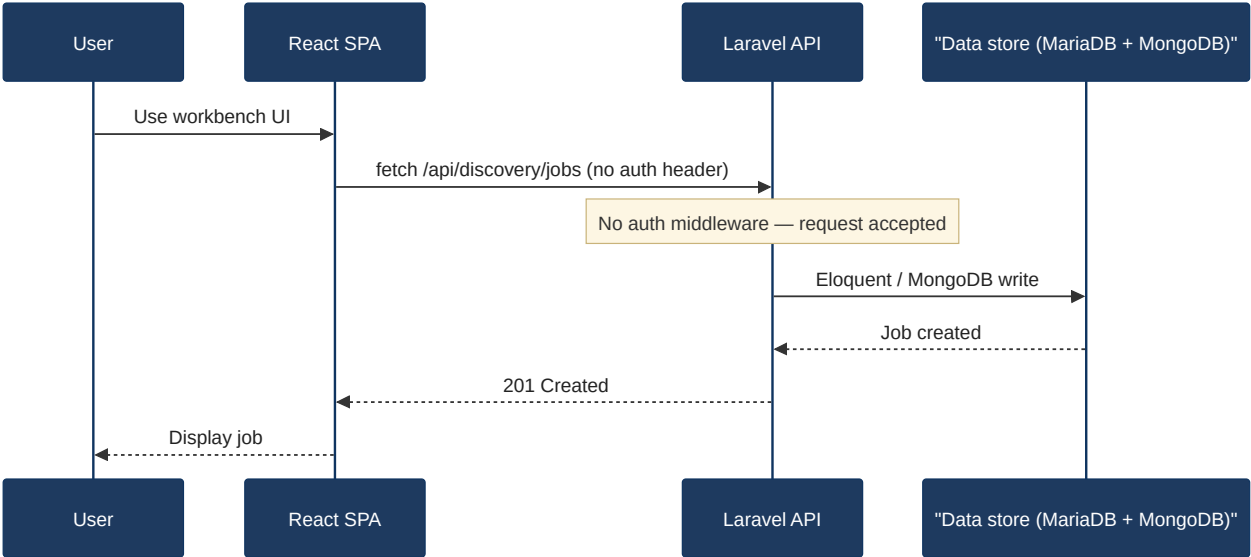
Not observed: SQL injection, CSRF, SSRF.

6.3 OWASP Top 10 (2021) Coverage

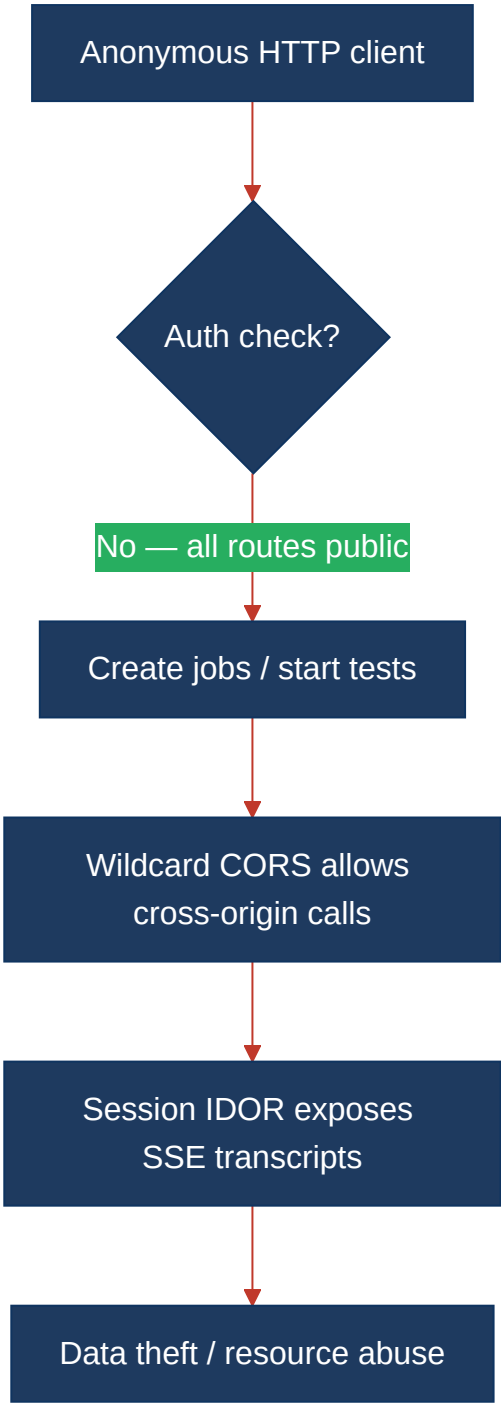
#	Category	Verdict	Evidence / Note
6.1	Broken Access Control	CRITICAL	All 19 API routes unauthenticated; session stream IDOR — §6.2
6.2	Cryptographic Failures	MEDIUM	Hardcoded <code>secret</code> DB password in compose/CI — §6.2 Default Credentials
6.3	Injection	MEDIUM	No SQL/shell injection; <code>extract(\$request->all())</code> variable injection — §6.2
6.4	Insecure Design	MEDIUM	No rate limiting on test-start/import endpoints — §6.2
6.5	Security Misconfiguration	HIGH	Wildcard CORS, APP_DEBUG=true, missing headers, HTTP default — §6.2, FS5
6.6	Vulnerable and Outdated Components	CLEAN	<code>npm audit</code> 0 CVEs; Laravel 12 / React 19 current
6.7	Identification and Authentication Failures	CRITICAL	No auth middleware, guards, or MFA — §6.2
6.8	Software and Data Integrity Failures	MEDIUM	CI has no <code>npm audit</code> or SAST step — §6.5 Actions
6.9	Security Logging and Monitoring Failures	MEDIUM	No audit log for API access; only <code>console.log</code> in dev-api
6.10	Server-Side Request Forgery (SSRF)	CLEAN	Not observed — no outbound HTTP from user input
6.11	Other Security Reviews	MEDIUM	Exposed DB ports; unauthenticated bulk-import — §6.2
6.12	DevSecOps Security Assessment	MEDIUM	CI runs PHPUnit + build only; no dependency scan or SAST

6.4 Diagrams

Auth / request trust boundary



Top security risk flow



Improvement roadmap



6.5 Actions Required

Finding	Action	Rating	Priority
---------	--------	--------	----------

Missing API Authentication	Add Sanctum/JWT auth middleware to all routes in <code>backend/routes/api.php</code> ; add policy checks in 6 controllers; mirror or remove unauthenticated dev-api.	HIGH RISK	CRITICAL
Permissive CORS	Replace <code>allowed_origins: ['*']</code> in <code>backend/config/cors.php</code> and default <code>cors()</code> in <code>dev-api/src/server.js</code> with env-driven allow-list.	HIGH RISK	HIGH
Session Stream IDOR	Bind <code>session_id</code> to <code>resource_id</code> + user in <code>RealTimeTestService</code> ; validate in <code>StreamController</code> and dev-api <code>streamSession()</code> .	HIGH RISK	HIGH
Unauthenticated Bulk Import	Remove or protect <code>dev-api /bulk-import</code> with auth, schema validation, and rate limiting.	HIGH RISK	HIGH
PHP extract() Variable Injection	Replace <code>extract(\$filters)</code> in <code>LegacyReportController</code> with typed DTO; refactor <code>LegacyDataMapper</code> to explicit key access.	MODERATE	MEDIUM
Default Credentials & Debug Mode	Remove hardcoded passwords from <code>docker-compose.yml</code> and <code>.env.example</code> ; set <code>APP_DEBUG=false</code> outside local.	MODERATE	MEDIUM
Missing Security Headers	Add CSP, HSTS, X-Frame-Options, and X-Content-Type-Options in <code>docker/nginx/default.conf</code> .	MODERATE	MEDIUM
No Rate Limiting	Apply Laravel <code>throttle</code> middleware and <code>express-rate-limit</code> on POST/start endpoints.	MODERATE	MEDIUM
Exposed Database Ports	Remove <code>3306:3306</code> and <code>27017:27017</code> host mappings from <code>docker-compose.yml</code> .	MODERATE	MEDIUM
FS5 — HTTP default & no CSP	Enforce HTTPS <code>VITE_API_URL</code> in production; add CSP to <code>frontend/index.html</code> or nginx.	MODERATE	MEDIUM
DevSecOps — no dependency scan in CI	Add <code>npm audit --audit-level=high</code> and <code>composer audit</code> steps to <code>.github/workflows/ci.yml</code> .	MODERATE	MEDIUM
No Security Audit Logging	Add structured audit log for resource create/start/stream events in Laravel.	MODERATE	MEDIUM

6.6 Expected Outcomes

- API authentication and authorization eliminate anonymous job creation, test execution, and transcript access.
- CORS allow-listing and session IDOR fixes prevent cross-site abuse and unauthorized SSE replay.
- Security headers, rate limiting, and closed database ports reduce DoS, clickjacking, and direct DB compromise risk.
- CI dependency scanning (`npm audit` , `composer audit`) catches future CVEs before merge.
- Replacing `extract()` with typed access removes a latent variable-injection footgun in legacy report code.